

Universidade do Minho
Escola de Engenharia

Otimização de Interrogações Apache Spark SQL para Geração Automática de Relatórios do Deucalion

Análise de Dados de Elevado Desempenho Trabalho Prático I

Grupo 12

Humberto Gil Azevedo Sampaio Gomes PG59806

José António Fernandes Alves Lopes PG59807

José António Costa Soares PG60277

19 de março de 2026

Resumo

A equipa do Deucalion automatizou a geração de relatórios estatísticos utilizando Apache Spark SQL [2]. Porém, devido à falta de otimização inicial, os tempos de execução tornaram-se excessivamente elevados, justificando a necessidade de melhorar o desempenho do código.

Através de uma metodologia iterativa, baseada na análise de planos de execução de interrogações e na recolha de métricas de desempenho, foram desenvolvidas e avaliadas quatro implementações otimizadas em Apache Spark. Além disso, foram realizados portes para PostgreSQL e DuckDB, com o objetivo de identificar a solução mais eficiente para os *datasets* de *jobs* de pequena dimensão (73 MiB).

Os resultados demonstraram que a implementação **FastLoad** atingiu um *speedup* de $28\times$ em relação à versão original, porém sacrificando alguma flexibilidade no esquema dos ficheiros do *dataset*. A solução em DuckDB superou todas as alternativas, alcançando um *speedup* de $40.5\times$.

1 Introdução

O Deucalion é um supercomputador cofinanciado pela Fundação para a Ciência e Tecnologia (FCT) e pelo projeto EuroHPC. Apresenta uma arquitetura heterogénea, integrando nós com processadores Arm (Fujitsu A64FX) e x86 (AMD EPYC), bem como nós com aceleradores gráficos (GPUs NVIDIA A100) [1]. A equipa do Deucalion produz relatórios periódicos com estatísticas mensais, trimestrais e anuais sobre o número de horas utilizadas e o número de *jobs* bem e mal-sucedidos, discriminados por partição (Arm, x86 e GPU), bem como quantos destes têm origem em projetos EuroHPC.

A equipa do Deucalion automatizou a geração destes relatórios utilizando o Apache Spark [2], uma *framework* para análise de dados em larga escala. O gerador de relatórios consiste num *script* Python que executa várias interrogações SQL sobre o *dataset* de *jobs* do Deucalion, seguidas da exportação dos resultados em formato LaTeX. No entanto, a implementação das interrogações foi feita com pouca atenção ao seu desempenho, resultando em código pouco eficiente e a um aumento significativo no tempo de geração de relatórios à medida que o número de *jobs* anuais no Deucalion aumenta.

Assim, procurou-se otimizar as interrogações Apache Spark SQL para diminuir o tempo da geração de relatórios. Além disso, dada a pequena dimensão do *dataset* de *jobs* (~ 73 MiB para 2025) e o facto do Apache Spark ser mais adequado para o processamento de grandes volumes de dados, investigaram-se também outras ferramentas para a análise dos dados, como PostgreSQL [3] e DuckDB [4].

2 Metodologia

A otimização da geração de relatórios seguiu um processo iterativo: partiu-se de uma implementação inicial, recolheram-se e analisaram-se métricas do seu desempenho, identificaram-se ineficiências, e implementaram-se otimizações para as eliminar, reduzindo-se progressivamente os tempos de execução. A versão final, desenvolvida em Apache Spark, foi depois portada para PostgreSQL e DuckDB, permitindo uma comparação direta do desempenho das três plataformas.

Para garantir a correção das várias implementações, desenvolveu-se um *script* que compara, para todos os meses de 2025, os relatórios gerados por cada implementação com os da implementação original. Os resultados revelaram-se equivalentes, à exceção de pequenos erros de aritmética de vírgula flutuante.

A estratégia de otimização iterativa adotada exige a definição de uma metodologia para medições que permita comparar o desempenho das diferentes implementações. Em primeiro lugar, estabeleceu-se que todas as medições de desempenho se baseariam na geração do relatório do mês de dezembro de 2025, dado que tal exige a leitura de todos os ficheiros de *jobs* mensais nos quais o *dataset* se divide. De seguida, devido ao tamanho reduzido do *dataset* de *jobs*, descartaram-se análises multi-nó, realizando-se todos os testes num único nó Arm do Deucalion (partição **normal-arm**), equipado com um processador Fujitsu A64FX de 48 núcleos ARMv8.2-A a 2.0 GHz.

As medições para comparação entre implementações utilizaram 48 *threads*, valor que coincide com o número de núcleos do processador, maximizando assim o grau de paralelismo disponível. Por exemplo, o Apache Spark foi executado no modo local, onde um único processo integra o *driver* e um *executor* capaz de criar até 48 *threads* para processamento de tarefas. Foram ainda realizadas análises de escalabilidade forte para a melhor implementação em Apache Spark e para a solução em DuckDB, com o objetivo de avaliar a sua capacidade de explorar plenamente o paralelismo do sistema.

Para analisar o desempenho de cada implementação, além da análise dos planos de execução das diferentes interrogações, instrumentou-se cada implementação, e recolheram-se as métricas apresentadas na Tabela 1:

Métricas Gerais	Métricas do Apache Spark
• Tempo de geração do relatório; • Percentagem média de uso da CPU; • Número de <i>bytes</i> lidos e escritos para disco, lógico (inclui <i>page cache</i>) e físico; • Número de <i>bytes</i> e pacotes transmitidos e recebidos pela rede InfiniBand.	• Número total de <i>jobs</i>, <i>stages</i> e <i>tasks</i>, e quantos destes foram de facto executados (remoção dos <i>skipped</i>); • Número de <i>bytes</i> e registos lidos e escritos em operações de <i>shuffle</i>; • Os dados acima, ao nível de <i>jobs</i>, <i>stages</i> e <i>tasks</i> individuais.

Tabela 1: Métricas de desempenho recolhidas para identificação de ineficiências.

As métricas de sistema (uso de CPU, disco e rede) foram recolhidas diretamente do sistema de ficheiros */proc*, enquanto que as métricas do Apache Spark foram obtidas através da sua API de instrumentação [5]. Procurou-se ainda recolher métricas sobre o uso de memória, tanto ao nível da *Java Virtual Machine* (JVM) como do sistema. No entanto, os resultados revelaram-se inconsistentes devido à atuação do Garbage Collector da JVM, pelo que foram descartados.

De forma a reduzir a variabilidade nos valores das métricas, cada resultado apresentado corresponde à média de cinco execuções consecutivas. Verificou-se, em análises preliminares, uma melhoria progressiva no desempenho das implementações em Apache Spark, associada ao aquecimento (*warm-up*)

da JVM. Por esse motivo, antes da recolha de quaisquer medições, descartaram-se cinco execuções para aquecimento. Esta metodologia, embora permita demonstrar o melhor desempenho alcançável por cada implementação, não reflete um cenário realista, visto que os relatórios serão gerados sem qualquer aquecimento.

Desenvolveu-se um *script* para automatizar a recolha de medições e facilitar a reprodução dos resultados obtidos. Este *script* é ainda responsável pelo tratamento estatístico dos dados, descartando automaticamente as execuções para aquecimento e calculando a média e o desvio padrão das diversas métricas para as restantes. Também para possibilitar a replicação dos resultados, a tabela 2 apresenta as versões do *software* utilizado:

<i>Software</i>	Versão
Sistema Operativo	Rocky Linux 8.5
Kernel Linux	4.18.0-348.23.1.el8_5.aarch64
Python	CPython 3.13.5
Apache Spark	4.1.1
<i>Runtime</i> Java	OpenJDK 21.0.8
PostgreSQL	16.1
DuckDB	1.5.0

Tabela 2: Versões do *software* relevante utilizado.

3 Implementações da Geração de Relatórios

3.1 Original

Esta implementação do gerador de relatórios, embora semelhante à versão original desenvolvida pela equipa do Deucalion, não é idêntica. Análises preliminares de desempenho revelaram que o tempo de execução despendido em código Python é negligenciável face ao tempo gasto no Apache Spark. Assim, a implementação original foi adaptada à infraestrutura existente de validação de resultados e medição de desempenho, o que implicou alterações ao código Python mas não às interrogações Spark. No entanto, uma pequena alteração a uma das interrogações Spark foi necessária para resolver um *crash* causado por dados mal formatados no *dataset*.

Nesta implementação, a geração de relatórios inicia-se com a leitura de todos os ficheiros *Comma-Separated Values* (CSV), um por mês, de um *dataset* anual de *jobs*. Os tipos de dados das colunas são inferidos automaticamente, e os dados são consolidados num único *data frame* que agrega todos os *jobs* do ano. De seguida, são adicionadas várias colunas auxiliares ao *data frame*, das quais as mais relevantes estão descritas na Tabela 3:

Coluna	Descrição
Period	Mês em que o <i>job</i> foi executado
COMPLETED	Estado de um <i>job</i> normalizado: COMPLETED ou FAILED
cluster	Tipo de nó onde o <i>job</i> foi executado, derivado da partição: ARM, AMD, GPU
Agency	Agência que financia o projeto do <i>job</i> , derivada do nome da conta: FCT, EHPC, LOCAL
totalJobSeconds	Número total de core-segundos / GPU-segundos consumidos pelo <i>job</i>

Tabela 3: Descrição das principais colunas auxiliares adicionadas pela implementação **Original**.

Por último, a implementação **Original** executa três vezes, uma para cada período reportado (mês, trimestre, e ano) a sequência de interrogações Spark descritas abaixo, obtendo, de forma quase direta, os valores do relatório a exportar:

1. Filtrar os *jobs* pelo período a reportar e agrupá-los por COMPLETED e cluster, de modo a contar os *jobs* bem e mal-sucedidos por partição;
2. Para cada partição (três interrogações: ARM, AMD e GPU), filtrar os *jobs* não-locais da partição no período a reportar. O resultado é, desnecessariamente, agrupado por cluster para calcular o número total de segundos de computação utilizados em cada partição;
3. Filtrar os *jobs* não-locais no período a reportar e agrupá-los por cluster, de modo a contar o número de *jobs* por partição;
4. Esta interrogação é equivalente a filtrar os *jobs* do projeto EuroHPC no período a reportar e a agrupá-los por cluster, para contar o número de *jobs* EuroHPC por partição. Na prática, está mal otimizada e executa outras operações redundantes (ver abaixo);
5. Semelhante ao ponto 4, mas contabilizando os segundos de computação de *jobs* EuroHPC por partição.

Considere-se, como exemplo a implementação da interrogação 4 abaixo e o seu plano de execução lógico (Figura 1), tanto para o período de um ano como de um mês:

```
... = self.data.filter(F.col('Period').isin(months)) \
                .groupBy(['Agency', 'cluster']) \
                .count() \
                .orderBy('Agency') \
                .filter(F.col('Agency') == 'EHPC') \
                .collect()
```

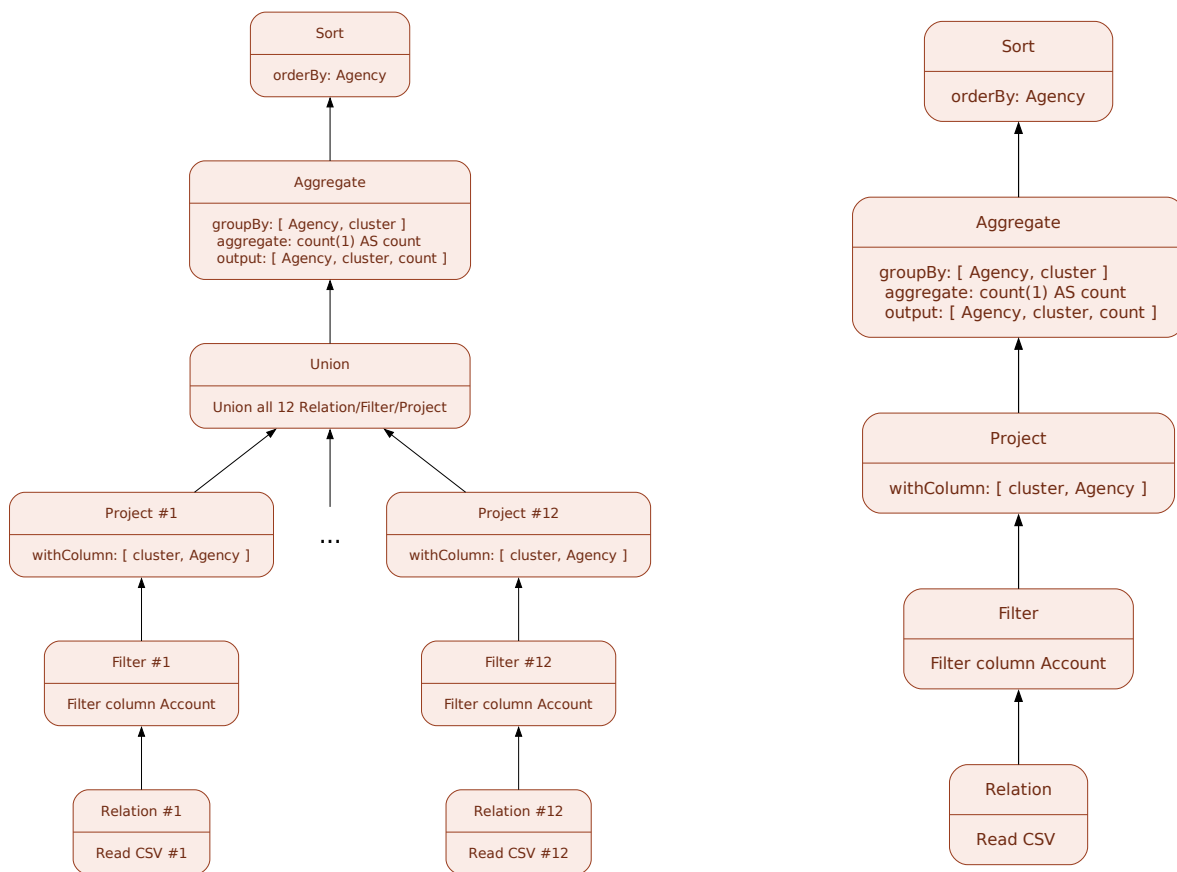


Figura 1: Planos de execução da 4ª interrogação Apache Spark SQL da implementação **Original**, para um período em análise de um ano (esquerda) e um mês (direita).

Como se pode observar, o otimizador do Apache Spark SQL aplicou várias otimizações típicas à interrogação, como a reordenação de operações e a substituição do filtro baseado na coluna **Agency** por outro baseado em **Account**. Contudo, a otimização mais notória foi a eliminação total do filtro por **Period**: uma vez que esta coluna assume um valor constante para cada ficheiro, o otimizador consegue filtrar os dados lendo apenas os ficheiros relevantes. Ainda assim, o otimizador não eliminou todas as ineficiências nesta interrogação. Por exemplo, manteve uma ordenação desnecessária por **Agency** que, no código da interrogação, é seguida da eliminação de registos com todos menos um valor de **Agency**.

Nas restantes interrogações desta implementação, o otimizador exibe um comportamento semelhante: consegue aplicar otimizações notáveis, mas não elimina todas as ineficiências. Contudo, o modelo de programação do Apache Spark não permite que o otimizador reduza o número de *actions* executadas (por exemplo, operações `collect`), das quais a implementação **Original** executa 21. Assim, o foco dos autores não se centrou na otimização das interrogações, mas sim na sua abstração face aos parâmetros de saída do relatório: não é necessária uma relação quase 1:1 entre as interrogações e os resultados a gerar. Só deste modo será possível reduzir o número de interrogações e de *actions*.

3.2 MultiPass

A leitura do *dataset* na implementação **MultiPass** é semelhante à da **Original**. No entanto, deixaram de ser geradas algumas colunas auxiliares que não eram utilizadas. Embora estas alterações, devido à otimização dos planos de execução, não afetem o desempenho da aplicação, contribuem para a melhoria da qualidade do código ao eliminar código redundante.

Após a leitura do *dataset*, esta implementação do gerador de relatórios executa, para cada período reportado (mês, trimestre, e ano), uma única interrogação Apache Spark SQL. Esta isola os *jobs* do período em questão, e agrupa-os por tuplos (**cluster**, **Agency**, **COMPLETED**), contando o número de *jobs* e tempo computacional associado a cada tuplo. O resultado é formado por registos como o da Tabela 4:

cluster	Agency	COMPLETED	COUNT(*)	sum(totalJobSeconds)
ARM	EHPC	COMPLETED	10	3600

Tabela 4: Exemplo de um registo resultante da interrogação na implementação **MultiPass**.

Ao iterar pelo resultado da interrogação, caso a implementação **MultiPass** encontre este registo, saberá que deve adicionar 10 ao número de *jobs* bem-sucedidos, ao número de *jobs* na partição Arm e ao número de *jobs* EuroHPC, bem como uma hora (3600 segundos) ao total de horas do projeto EuroHPC e na partição Arm.

A Figura 2 apresenta os planos de execução da interrogação descrita. Verifica-se, mais uma vez, que o otimizador filtra os *jobs* de um dado período através da leitura exclusiva dos ficheiros CSV correspondentes aos meses desse intervalo. No entanto, ainda são executadas três *actions* durante a geração do relatório, valor que pode ser ainda ser reduzido.

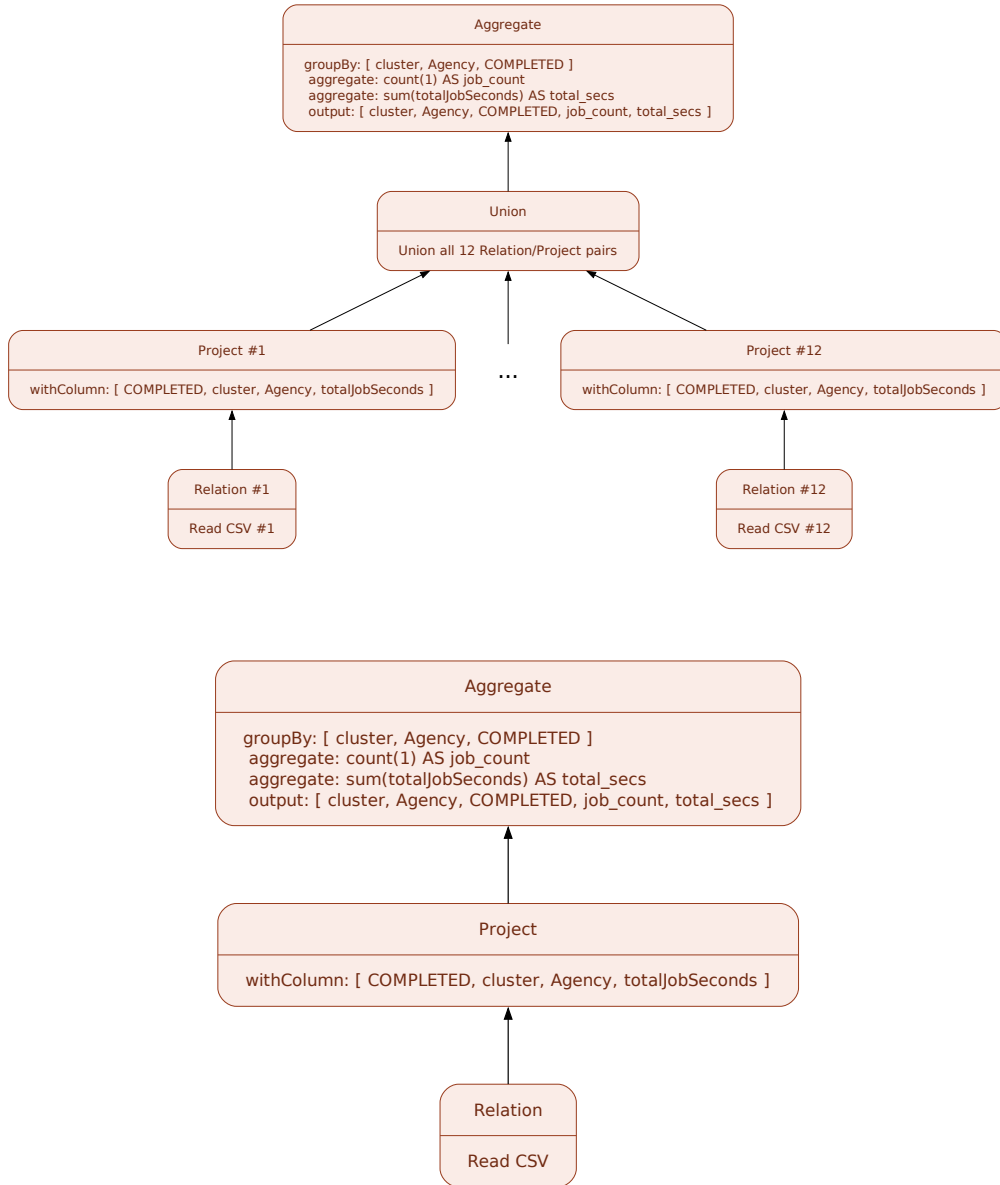


Figura 2: Planos de execução da interrogação Apache Spark SQL na implementação **MultiPass**, para um período em análise de um ano (cima) e um mês (baixo).

3.3 SinglePass

Ao contrário da **MultiPass**, esta implementação executa apenas uma *action* na etapa de processamento do *dataset*. O seu funcionamento é semelhante: agrupa os *jobs* por tuplos (**Period**, **cluster**, **Agency**, **COMPLETED**) numa única operação. Depois, a informação do mês associada a cada grupo de *jobs* (**Period**) permite determinar para quais dos períodos (mês, trimestre ou ano) esse deve ser contabilizado.

O plano de execução desta interrogação, semelhante ao de **MultiPass**, está representado na Figura 3. Análises de desempenho preliminares mostraram que a maior parte do tempo de execução tinha

origem na leitura do *dataset*, tendo-se esse tornado o próximo foco da otimização.

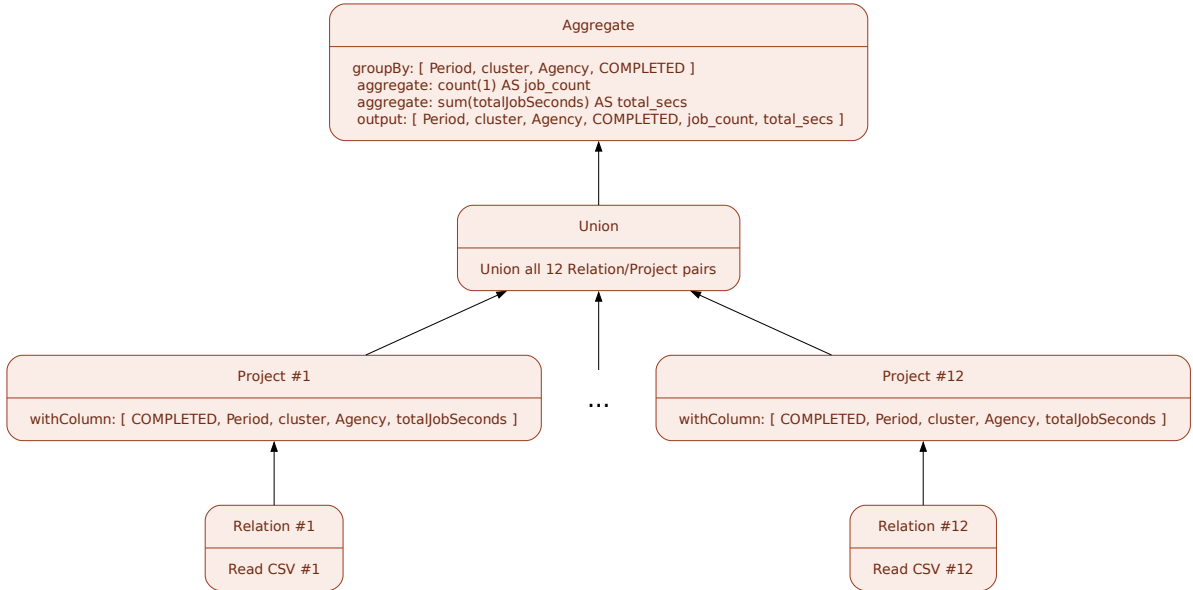


Figura 3: Plano de execução da interrogação Apache Spark SQL na implementação **SinglePass**.

3.4 FastLoad

Esta implementação do gerador de relatórios distingue-se da **SinglePass** apenas na forma como se realiza a leitura do *dataset*.

Em primeiro lugar, todas as implementações anteriores inferem o esquema dos ficheiro CSV (colunas e tipos de dados) com base no seu conteúdo, uma operação que requer um passe adicional de processamento dos dados [6]. A implementação **FastLoad** define manualmente o esquema dos ficheiros, eliminando esse trabalho adicional, embora torne a *query* menos flexível: se forem adicionadas novas colunas aos ficheiros, esta implementação terá de ser atualizada para as suportar.

Além disso, esta implementação também tira proveito das funcionalidades de leitura de ficheiros em *bulk* do Apache Spark. Assim, é possível ler todos os ficheiros do *dataset* numa única operação, evitando uma união no plano de execução da interrogação, como mostra a Figura 4:

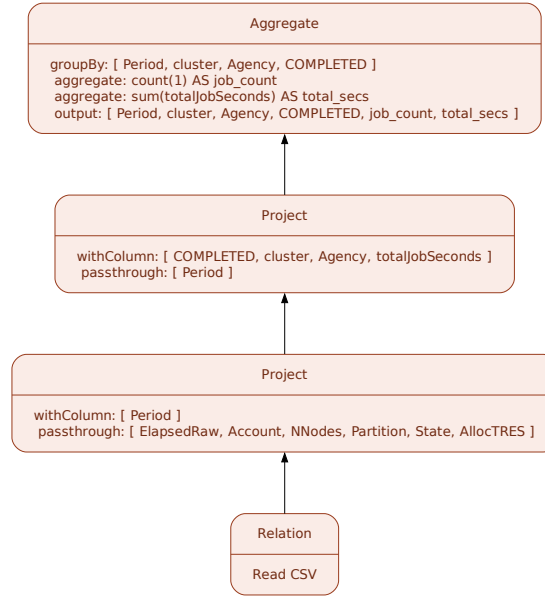


Figura 4: Plano de execução da interrogação Apache Spark SQL na implementação **FastLoad**.

3.5 PostgreSQL

Para avaliar o desempenho do Apache Spark em comparação com outras plataformas, a implementação melhor otimizada (**FastLoad**) foi portada para PostgreSQL e DuckDB. Contudo, o PostgreSQL apresenta limitações significativas na leitura de ficheiros CSV, nomeadamente a incapacidade de importação de ficheiros em *bulk*. Assim, a implementação **PostgreSQL** aproxima-se mais da abordagem **SinglePass** do que da **FastLoad**.

Neste contexto, cada ficheiro tem de ser importado na sua totalidade para uma tabela em memória, sem possibilidade de filtrar colunas durante o processo. Só após esta leitura completa é possível isolar os campos relevantes (ver Tabela 3). Já a geração do relatório mantém-se equivalente às implementações **FastLoad** e **SinglePass**: uma única interrogação PostgreSQL agrega toda a informação necessária para produzir os valores finais do relatório.

3.6 DuckDB

A geração de relatórios com DuckDB consiste num porte direto da implementação **FastLoad**. Em primeiro lugar, todos os ficheiros CSV são lidos em *bulk* e processados, gerando uma tabela em memória que contém apenas as colunas necessárias para a geração de relatórios (ver Tabela 3). De seguida, esta tabela é processada de forma análoga à implementação **SinglePass**, recorrendo a uma interrogação DuckDB.

4 Resultados Obtidos

4.1 Comparação das Implementações

O desempenho das várias implementações desenvolvidas foi medido, e encontra-se apresentado na Figura 5 abaixo. Importa realçar que, de acordo com a metodologia adotada, não foram contabilizados os tempos de inicialização do *cluster* do Spark (quando aplicável).

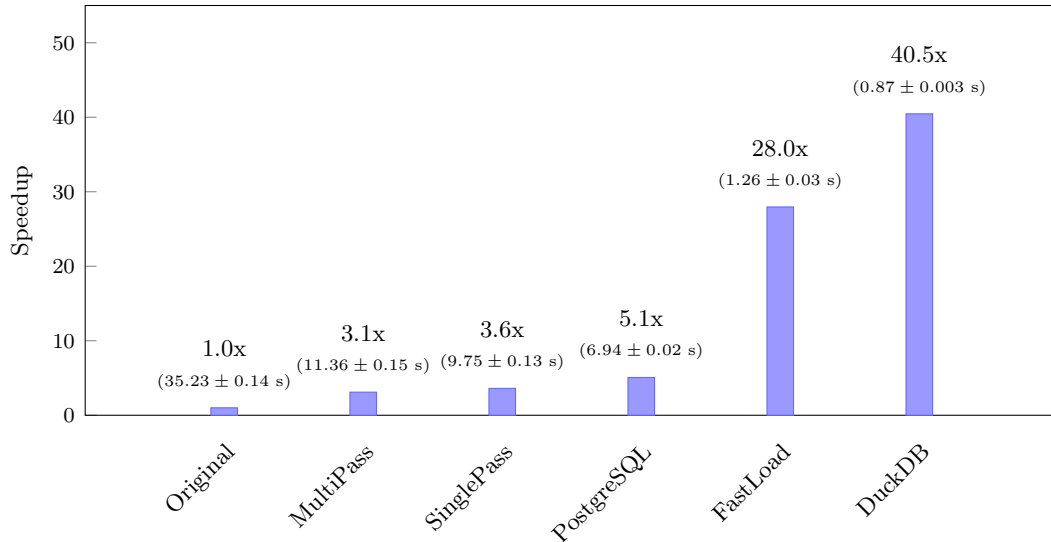


Figura 5: *Speedups* relativos e tempos de execução (média $\pm$ desvio padrão) das diferentes implementações de geradores de relatórios.

No que diz respeito às implementações baseadas em Apache Spark, verifica-se que, como esperado, o desempenho melhora à medida que são aplicadas mais otimizações. Contudo, importa compreender as razões subjacentes a esta melhoria. Para tal, analisaram-se algumas métricas relevantes recolhidas durante a execução das várias implementações, apresentadas na Tabela 5.

Implementação	Tempo (s)	Jobs	Estágios	Tarefas	Leituras de Disco Lógicas (MiB)
Original	35.23 ± 0.14	72	72	72	908.350
MultiPass	11.36 ± 0.15	30	30	76	196.280
SinglePass	9.75 ± 0.13	26	26	61	151.539
FastLoad	1.26 ± 0.03	2	2	25	73.835

Tabela 5: Comparação de métricas de instrumentação Spark entre as diferentes implementações. Os números de estágios e tarefas referem-se aos executados (excluem-se os *skipped*).

A análise das métricas de execução revela as principais causas responsáveis para a melhoria de desempenho entre as implementações **Original**, **MultiPass** e **SinglePass**: observa-se uma diminuição significativa no número de *jobs* e de estágios, mas sobretudo na quantidade de leituras de disco efetu-

adas. Como estas implementações não recorrem a qualquer mecanismo de *caching*, cada interrogação Spark SQL implica a leitura de um ou mais ficheiros CSV. Assim, ao diminuir o número de interrogações, como fazem as otimizações aplicadas, diminui-se também a quantidade de dados lidos e o número de pedidos ao sistema operativo. A diferença de desempenho entre as implementações **MultiPass** e **SinglePass** é menor, uma vez que o trabalho na análise inicial do esquema dos ficheiros CSV se começa a tornar muito significativo, representando cerca de metade das leituras efetuadas.

Já o ganho significativo de $7.8\times$ entre as implementações **SinglePass** e **FastLoad** explica-se de forma distinta. Embora se registre uma redução acentuada no número de *jobs* e estágios, verifica-se que o número médio de tarefas por estágio aumenta de 2.4 para 12.5. Isto sugere que esta implementação consegue explorar de forma mais eficiente o paralelismo disponibilizado pelos vários núcleos do processador. Infelizmente, foram obtidas medições inválidas da taxa de utilização da CPU (com valores negativos), o que impede a confirmação de uma eventual melhoria na eficiência da sua utilização. Importa referir, no entanto, que, apesar do menor tempo de execução desta versão, a sua adoção impede a modificação do esquema dos ficheiros CSV sem alteração do código.

O DuckDB, por ser uma ferramenta desenvolvida em C++ (o que lhe confere menor *overhead*) e ter um maior foco em *datasets* mais pequenos, apresentou um melhor desempenho. A geração de relatórios com tecnologias que não Apache Spark teve um carácter meramente exploratório, pelo que não se aprofundou a análise das razões subjacentes ao melhor desempenho da implementação DuckDB.

Importa ainda realçar que o desempenho inferior observado na solução PostgreSQL se deveu, sobretudo, ao tempo necessário para a importação inicial dos dados. Num cenário real, em que os dados já estivessem persistidos na base de dados, o tempo de resposta às interrogações seria significativamente mais competitivo.

4.2 Análises de Escalabilidade

Foi realizada uma análise de escalabilidade forte para comparar a melhor implementação em Apache Spark (**FastLoad**) com a solução em DuckDB, e averiguar se estas conseguem tirar proveito do grau de paralelismo oferecido pelos 48 núcleos do processador Fujitsu A64FX. Os resultados obtidos são apresentados na Figura 6:

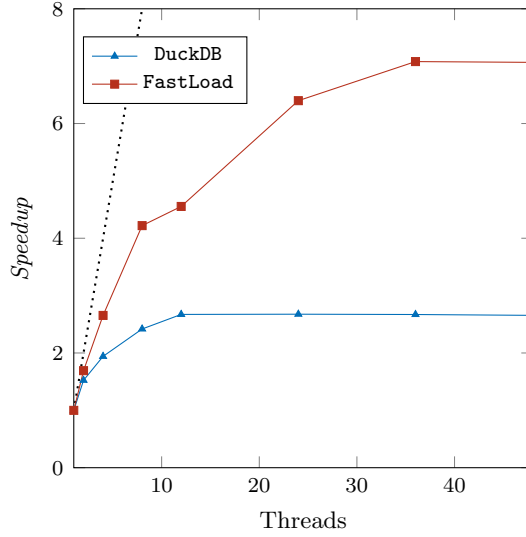


Figura 6: Resultados das análises de escalabilidade forte das implementações FastLoad e DuckDB. A linha a tracejado representa o *speedup* linear (ideal).

Observa-se que a solução em Apache Spark escala melhor com o aumento do número de *threads*, atingindo um *speedup* máximo de $7.1\times$ em relação à execução *single-threaded* do mesmo *workload*. Este valor é superior ao *speedup* máximo de $2.7\times$ obtido com DuckDB. Ainda assim, a implementação DuckDB supera a FastLoad em tempo de execução puro. No entanto, nenhuma das implementações se aproxima do *speedup* linear ideal.

Os autores apresentam duas teorias para explicar os resultados: a arquitetura do Apache Spark está melhor concebida para tirar proveito de um maior número de *threads*, e / ou o DuckDB é tão eficiente que se aproxima dos limites computacionais do sistema (por exemplo, largura de banda de memória ou problemas de contenção no *kernel* Linux). Contudo, avaliar estas hipóteses exigiria uma análise de desempenho muito mais detalhada, fora do âmbito deste estudo.

O que se observa é que a percentagem de tempo do *executor* do Spark despendida na realização de tarefas ($1 - \text{Idle ratio}$) diminui à medida que o número de *threads* aumenta, como ilustra a figura 7, sugerindo possíveis aumentos no *overhead* de sincronização e potenciais problemas de desbalanceamento de carga.

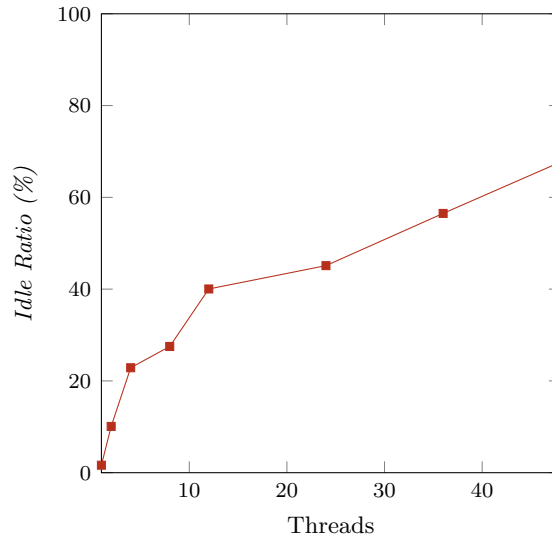


Figura 7: Evolução do *idle ratio* da implementação **FastLoad** com o aumento do número de *threads*.

4.3 Restantes Tentativas de Otimização

Ao longo do processo de otimização, foram exploradas diversas estratégias que, embora potencialmente benéficas, não resultaram em melhorias de desempenho mensuráveis para o caso de estudo em análise.

Em primeiro lugar, não se verificou qualquer benefício na utilização dos mecanismos de *cache* e persistência do Apache Spark, uma vez que as implementações mais eficientes não reutilizam dados: o fluxo de processamento é linear, e os ficheiros CSV são lidos e processados apenas uma vez. Da mesma forma, não se registaram ganhos com o aumento da memória atribuída ao *driver* e ao *executor* (parâmetros `driver.memory` e `executor.memory`), dado que o *dataset* cabe integralmente em memória e não ocorrem *shuffles* significativos para disco (todos os *shuffles* são na ordem dos poucos KiB).

Ao contrário das otimizações anteriores, que não encontram justificação teórica para este caso de estudo, a estratégia mais promissora avaliada foi o ajuste do tamanho das partições dos dados (parâmetro `spark.sql.files.maxPartitionBytes`). Sem qualquer ajuste, o número máximo de tarefas geradas pelo Spark na implementação **FastLoad** é 24. Reduzir o tamanho das partições para 2 MiB permite aumentar o número de tarefas para 43, possibilitando um maior grau de paralelismo. Contudo, o maior número de tarefas não se traduziu numa melhoria de desempenho, mesmo quando testados outros tamanhos de partição (1 MiB e 3 MiB).

Por outro lado, identificou-se uma potencial otimização. Para gerar o relatório de um determinado mês, basta considerar os ficheiros de *jobs* até essa data. Implementações como **DuckDB**, no entanto, leem sempre todos os ficheiros de *jobs*, independentemente do mês do relatório. Embora esta abordagem não afete o desempenho no mês em estudo (dezembro, já que todos os ficheiros têm de ser lidos), pode revelar-se ineficiente para os restantes meses.

4.4 Problemas Metodológicos

Como já foi referido, a metodologia adotada, devido às suas execuções para aquecimento, não reflete um cenário realista. No entanto, existem outros problemas a considerar. Em primeiro lugar, uma análise mais detalhada do desempenho de execuções consecutivas da implementação **FastLoad** revelou que cinco execuções não são suficientes para aquecer JVM até ao seu estado ótimo. A figura 8 mostra que a média das cinco medições usualmente recolhidas é superior ao desempenho máximo que a JVM pode oferecer (com uma diferença de $\sim 20\%$). Contudo, atingir esse desempenho exigiria um número excessivamente elevado de execuções para aquecimento (mais de 30), o que se torna impraticável.

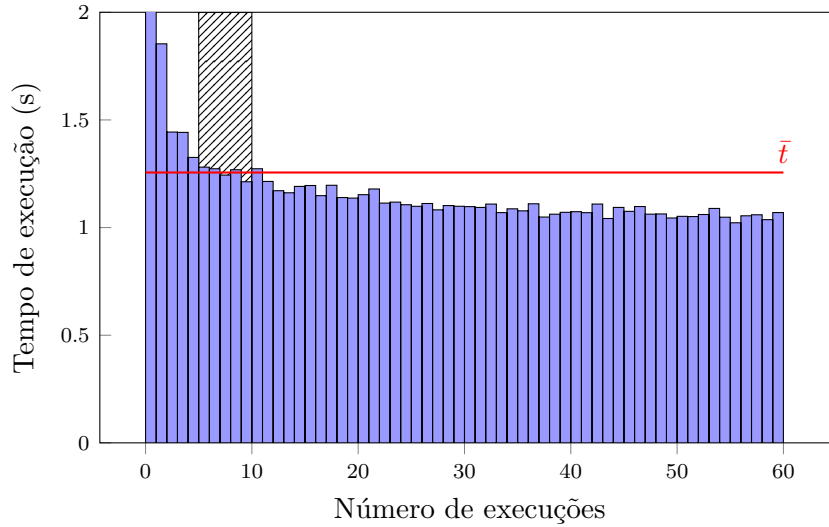


Figura 8: Tempos de execuções sucessivas de **FastLoad**, com medições usualmente recolhidas (5 medições após 5 execuções para aquecimento) e média das mesmas representadas. Para uma melhor legibilidade, o tempo da primeira execução não é visível na escala do gráfico.

Além disso, devido à existência da *page cache*, a exclusão das primeiras execuções para aquecimento é também relevante para aplicações que não são baseadas na JVM, como a geração de relatórios com PostgreSQL e DuckDB. Na primeira execução, o *dataset* é lido a partir do disco e colocado na *page cache*, uma cache em memória do sistema operativo que otimiza operações de *input-output*. Já na segunda execução, o *dataset* será lido a partir da *page cache*, tornando irrelevante o desempenho do meio de armazenamento onde se encontra. Isto é possível porque o *dataset* cabe integralmente em memória.

A Tabela 6 mostra as leituras lógicas e físicas de disco das primeiras execuções de **FastLoad** e evidencia que, após a primeira execução, deixam de ocorrer leituras físicas de disco. Este comportamento é observado em todas as restantes implementações. No entanto, sem permissões de super-utilizador, não é possível limpar a *page cache* entre execuções, o que impede a eliminação deste efeito nos resultados.

	Leituras de disco lógicas (MiB)	Leituras de disco físicas (MiB)
Execução 1	87.69	73.44
Execução 2	73.84	0.00
Execução 3	73.84	0.00

Tabela 6: Leituras de disco lógicas (incluem *page cache*) e físicas (implicam acesso ao LustreFS) durante as primeiras execuções do gerador de relatórios **FastLoad**.

Em suma, a metodologia adotada não só é altamente irrealista, como também falha em capturar o desempenho máximo dos geradores de relatórios baseados em Apache Spark.

5 Conclusões

Em suma, foi possível otimizar significativamente a geração dos relatórios periódicos do Deucalion, recorrendo a métricas de sistema e às excelentes ferramentas de instrumentação do Apache Spark para comparação e avaliação das otimizações implementadas.

Utilizando Apache Spark, a implementação **FastLoad** permite alcançar um *speedup* de $28\times$, desde que se esteja disposto a abdicar da flexibilidade do reconhecimento automático do esquema dos ficheiros no *dataset*. Caso essa opção não seja viável, a implementação desenvolvida **SinglePass** oferece um *speedup* mais moderado, mas ainda relevante, de $3.6\times$.

A principal conclusão deste trabalho foi a importância crítica da seleção das ferramentas adequadas para cada tarefa. O Apache Spark, enquanto *framework* para tratamento de dados em larga escala, não é a solução mais eficiente para um *dataset* de apenas 73 MiB. Por exemplo, a implementação da geração de relatórios em DuckDB permitiu obter um *speedup* de $40.5\times$ em relação à implementação original em Spark.

No entanto, as medições de desempenho apresentadas não refletem um cenário realista, uma vez que incluem execuções para aquecimento que não ocorreriam em ambiente de produção. Além disso, estas medições não consideram outros fatores cruciais no desenho de sistemas de geração de relatórios. Por exemplo, seria pertinente avaliar se as garantias ACID (Atomicidade, Consistência, Isolamento e Durabilidade) proporcionadas por um sistema de gestão de bases de dados relacionais, como o PostgreSQL, poderiam trazer benefícios significativos em relação ao atual armazenamento dos *jobs* em ficheiros CSV, mesmo que isso implique um eventual compromisso no desempenho da geração de relatórios.

Financiamento

Este trabalho utilizou horas de computação atribuídas pela bolsa 2025.00010.HPCVLAB.UMINHO, financiada pela Fundação para a Ciência e a Tecnologia (FCT).

Referências

- [1] National Advanced Computing Network. “DEUCALION.” [rnca.fccn.pt](https://rnca.fccn.pt/en/deucalion). Accessed: Mar. 18, 2026. [Online.] Available: <https://rnca.fccn.pt/en/deucalion>
- [2] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: cluster computing with working sets,” in *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*, 2010, pp. 10–17.
- [3] M. Stonebraker and L. A. Rowe, “The design of POSTGRES,” in *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*, 1986, pp. 340–355.
- [4] M. Raasveldt and H. Mühleisen, “DuckDB: an Embeddable Analytical Database,” in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1981–1984.
- [5] Apache Software Foundation. “Monitoring and Instrumentation.” [spark.apache.org](https://spark.apache.org/docs/latest/monitoring.html). Accessed: Mar. 19, 2026. [Online.] Available: <https://spark.apache.org/docs/latest/monitoring.html>
- [6] Apache Software Foundation. “CSV Files.” [spark.apache.org](https://spark.apache.org/docs/latest/sql-data-sources-csv.html). Accessed: Mar. 19, 2026. [Online.] Available: <https://spark.apache.org/docs/latest/sql-data-sources-csv.html>

A Planos de Execução e Implementações

A.1 Original

A.1.1 Interrogação 1

```
... = self.data.filter(F.col('Period').isin(months)) \
               .groupBy('COMPLETED', 'cluster')      \
               .count()                               \
               .collect()
```

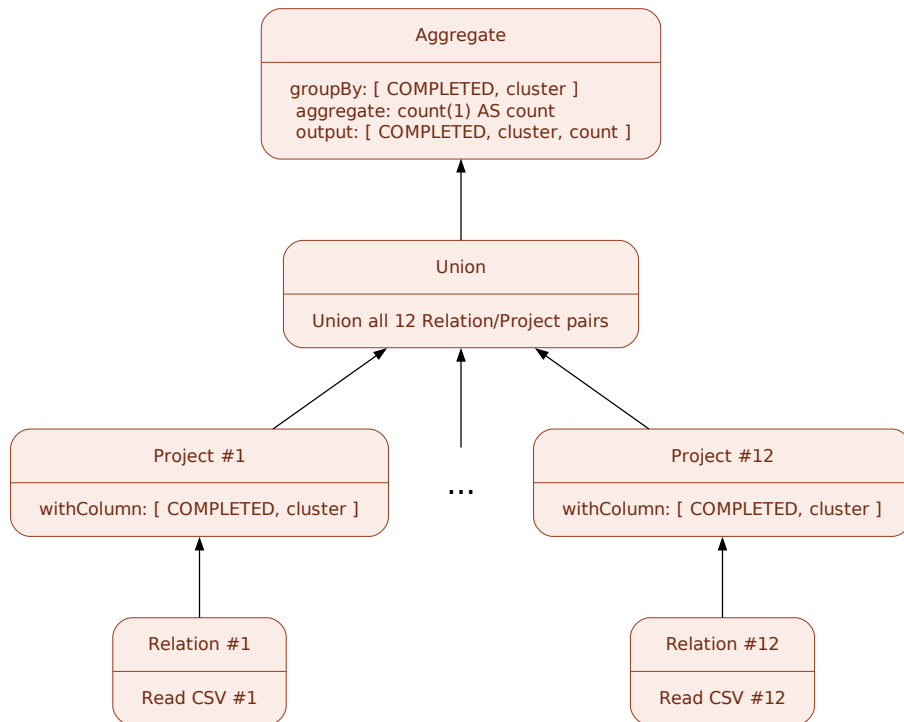


Figura 9: Plano lógico anual: Original Q1.

A.1.2 Interrogação 2

```
... = self.data.filter(F.col('Agency') != 'LOCAL') \
               .filter(F.col('Period').isin(months)) \
               .groupBy('cluster') \
               .sum('totalJobSeconds') \
               .filter(F.col('cluster') == cluster) \
               .collect()[0] \
               .asDict()['sum(totalJobSeconds)']
```

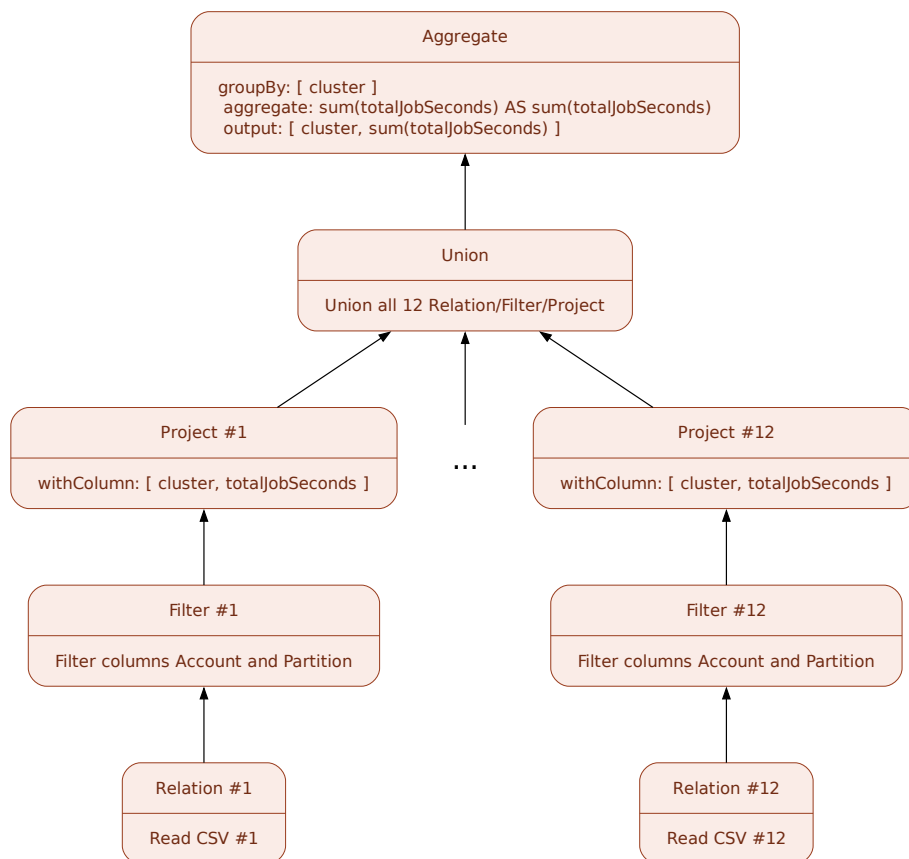


Figura 10: Plano lógico anual: Original Q2.

A.1.3 Interrogação 3

```
... = self.data.filter(F.col('Agency') != 'LOCAL') \
               .filter(F.col('Period').isin(months)) \
               .groupby('cluster') \
               .count() \
               .collect()
```

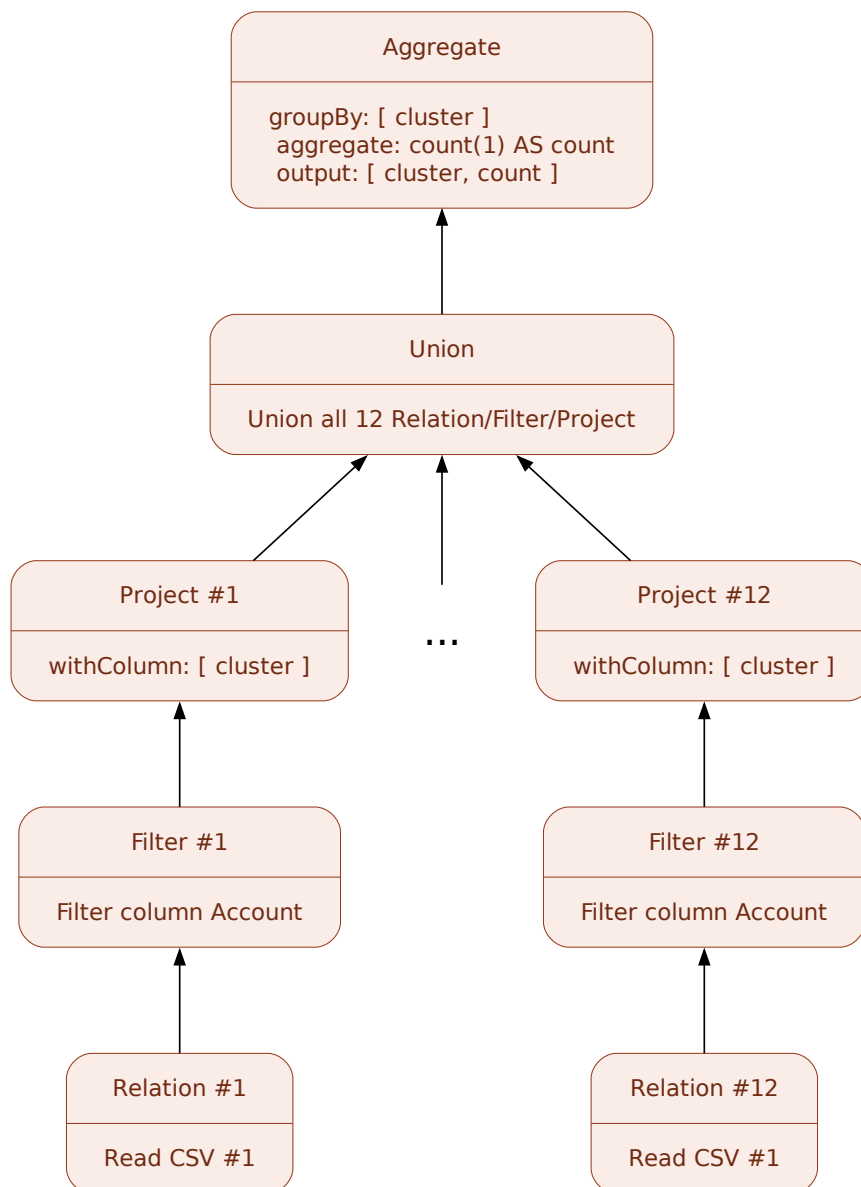


Figura 11: Plano lógico anual: Original Q3.

A.1.4 Interrogação 4

```
... = self.data.filter(F.col('Period').isin(months)) \
    .groupBy(['Agency', 'cluster']) \
    .count() \
    .orderBy('Agency') \
    .filter(F.col('Agency') == 'EHPC') \
    .collect()
```

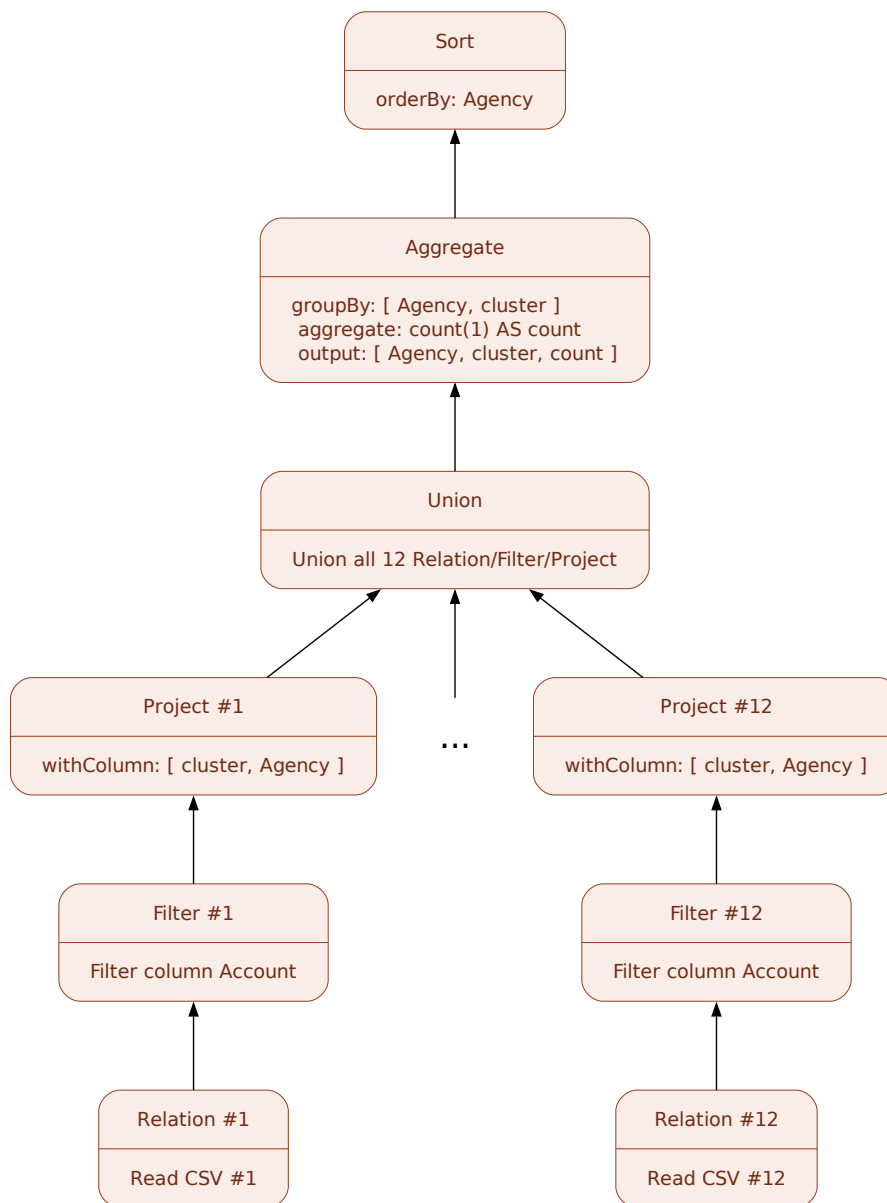


Figura 12: Plano lógico anual: Original Q4.

A.1.5 Interrogação 5

```
... = self.data.filter(F.col('Agency') == 'EHPC') \
               .filter(F.col('Period').isin(months)) \
               .groupby(['Agency', 'cluster']) \
               .sum() \
               .collect()
```

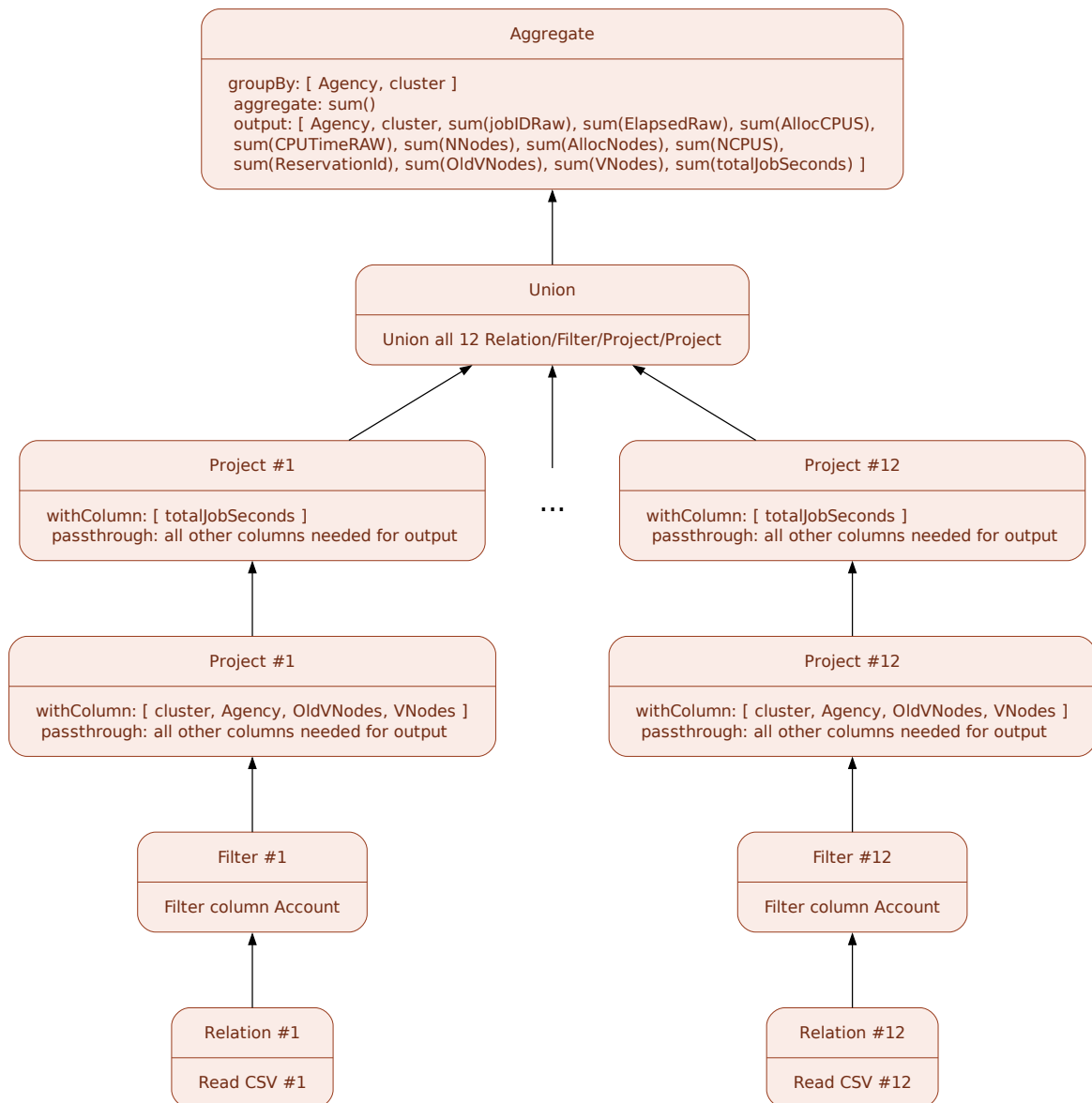


Figura 13: Plano lógico anual: Original Q5.

A.2 MultiPass

```
... = self.data.filter(F.col('Period').isin(months)) \
    .groupBy('cluster', 'Agency', 'COMPLETED') \
    .agg(
        F.count('*').alias('job_count'),
        F.sum('totalJobSeconds').alias('total_secs')
    ).collect()
```

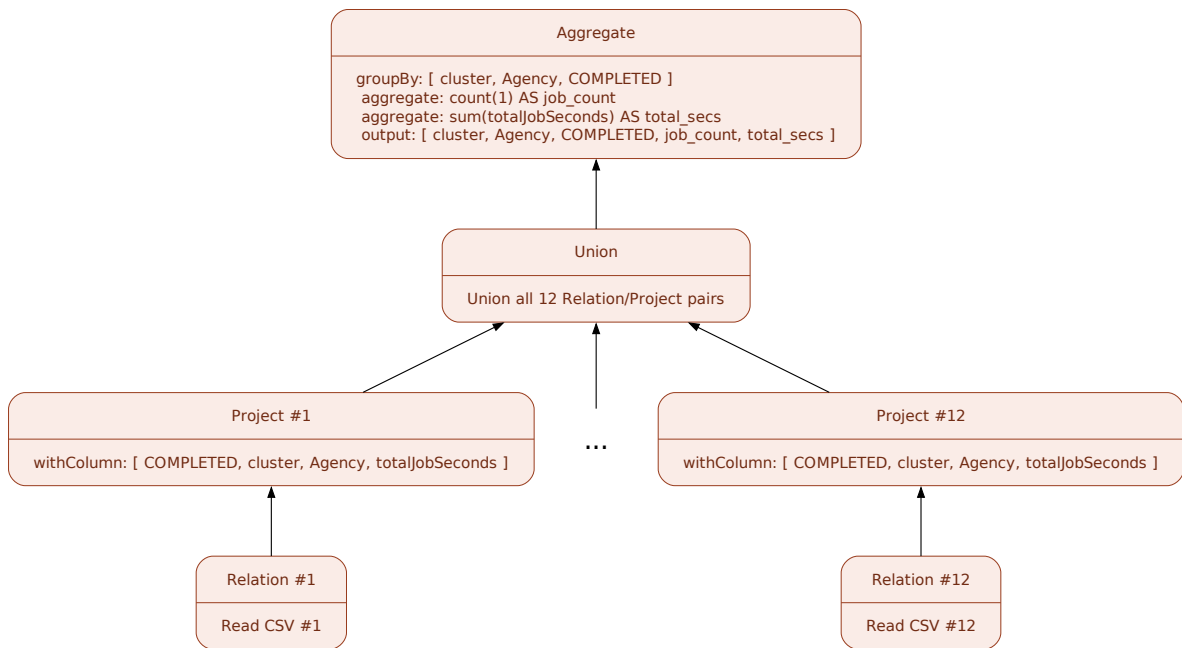


Figura 14: Plano de execução anual: MultiPass.

A.3 SinglePass

```
... = self.data.groupby('Period', 'cluster', 'Agency', 'COMPLETED') \
    .agg(
        F.count('*').alias('job_count'),
        F.sum('totalJobSeconds').alias('total_secs')
    ).collect()
```

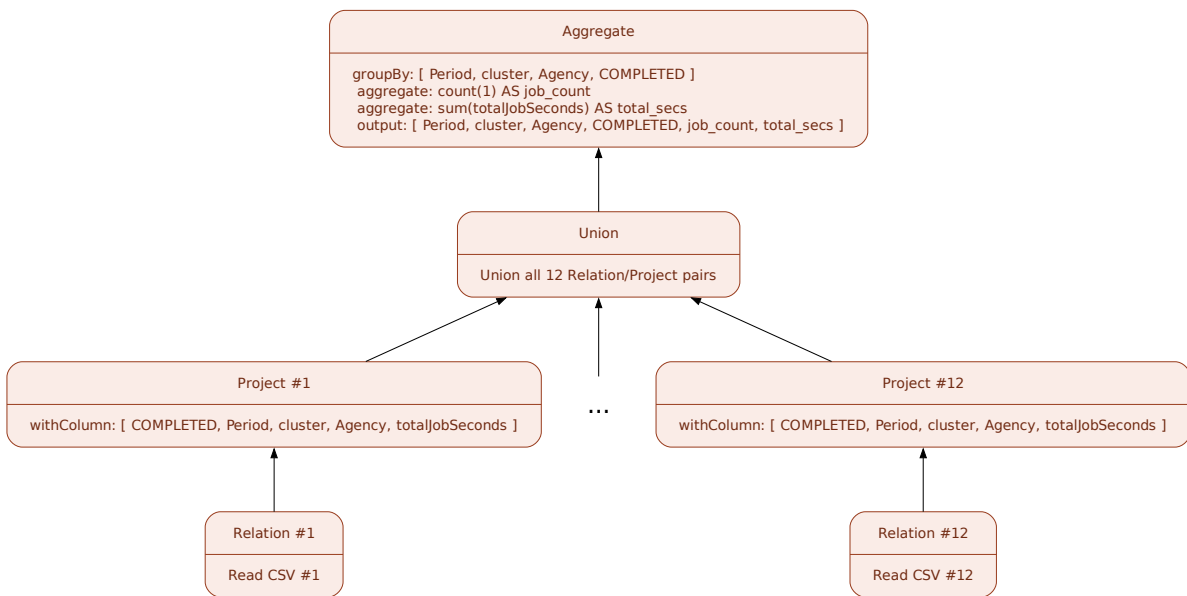


Figura 15: Plano de execução: SinglePass.

A.4 FastLoad

```
... = self.data.groupby('Period', 'cluster', 'Agency', 'COMPLETED') \
    .agg(
        F.count('*').alias('job_count'),
        F.sum('totalJobSeconds').alias('total_secs')
    ).collect()
```

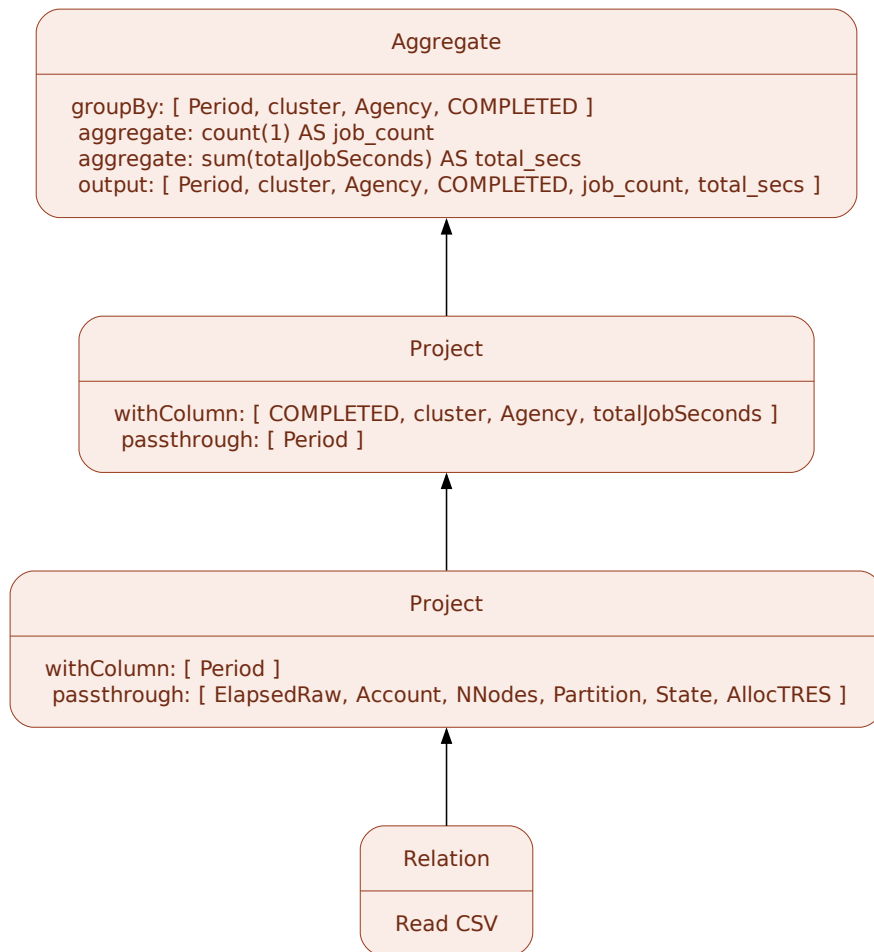


Figura 16: Plano de execução: FastLoad.